



A Course End Project Report on

Text Encryption and Decryption in Python

A9511 – Python Programming Laboratory

**Submitted by
Name**

**A.Kishore
25881A0591
D.Nithin
26881A0509**

Course Facilitator

**P.S.Madhavi
Assistant Professor**

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad Academic

Year 2025–26

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled “**Text Encryption and Decryption in Python**” is carried out by **A.Kishore** , **D.Nithin** with Roll Numbers **25881A0591** , **26881A0509** towards **A9511 - Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
P.S.Madhavi
AssistantProfessor,CSE

Signature of the
HOD
Dr .Y.Vijayalata,
Professor, CSE,
HOD and
Dean(Student
Mentoring),
Vardhaman
College of
Engineering

Contents

1	Introduction.....	3
2	Literature Review.....	4
3	Objectives.....	5
4	Methodology.....	6
4.1	Architectural Diagram.....	6
4.2	Hardware and Software Requirements.....	7
4.3	Working Principle.....	7
4.4	Software Implementation.....	9
5	Results and Discussion.....	11
6	Mapping POs and SDGs.....	13
6.1	Program Outcomes (POs) and (PSOs) Mapping.....	13
6.2	Sustainable Development Goals (SDGs) Mapping.....	14
7	Conclusion.....	15
	Bibliography.....	16

Abstract

The Text Encryption and Decryption System has been developed in Python using only its standard library, providing a simple, portable and menu-driven tool for protecting text messages. Users can type a message, choose one of the supported ciphers, provide the required key or shift value, and instantly obtain the encrypted or decrypted result on the console. The program correctly handles uppercase and lowercase letters, digits, spaces and punctuation, and reports clear error messages when the input, key or mode is invalid. Each algorithm is implemented as an independent pair of encrypt and decrypt functions, so the same logic that hides a message can also be used to recover it.

The backend implements five techniques: the Caesar cipher, the Vigenère cipher, a keyed monoalphabetic Substitution cipher, the Atbash cipher, and a byte-level XOR cipher whose output is encoded with Base64 so that it can be safely displayed and copied. Together they cover monoalphabetic, polyalphabetic and byte-oriented approaches, using modular arithmetic, string handling, loops, functions, exception handling and the base64 module. A modular design keeps the cipher logic separate from the user interface, improving readability, testing and reuse. The project helps learners understand the fundamentals of cryptography, compare the strengths and weaknesses of classical algorithms, and see why modern secure systems rely on far stronger standards.

Keywords: Text Encryption, Decryption, Cryptography, Python, Caesar Cipher, Vigenère Cipher, Substitution Cipher, Atbash Cipher, XOR, Base64, Information Security.

1 Introduction

In the digital age, enormous amounts of text data such as messages, emails, passwords, documents and financial details are stored and transmitted over networks every second. If this information is sent in plain form, anyone who intercepts it can read or modify it. Cryptography is the science of securing information by converting readable data, called plaintext, into an unreadable form, called ciphertext, using an algorithm and a key. The reverse process, which recovers the original message from the ciphertext, is known as decryption. Encryption therefore forms the foundation of confidentiality in modern communication systems.

The importance of encryption grows with the amount of sensitive data exchanged through social media, online banking, e-commerce and cloud services. The main goals of information security are confidentiality, integrity, authentication and non-repudiation, and encryption directly supports confidentiality by ensuring that only authorised parties who hold the correct key can understand a message. Even simple text encryption programs help students appreciate how keys, algorithms and data transformations work together, which is essential before moving on to advanced standards such as AES and RSA.

Classical ciphers are the historical ancestors of modern cryptography and are ideal for learning. The Caesar cipher shifts every letter by a fixed number of positions, the Atbash cipher reverses the alphabet, and the Substitution cipher replaces each letter using a keyed alphabet. The Vigenère cipher improves on these by using a repeating keyword so that the same letter can be encrypted differently at different positions. At the byte level, the XOR operation combined with a secret key and Base64 encoding shows how digital data is scrambled and represented as printable text. All of these can be expressed with modular arithmetic, string processing and simple loops.

Python is an excellent language for building such a tool because of its readable syntax, powerful string handling, built-in functions such as `ord()` and `chr()`, and the standard `base64` module. The purpose of this project is to design a menu-driven Python program that encrypts and decrypts text using several algorithms, validates user input, and demonstrates how each technique transforms a message. By running the program and studying its outputs, learners gain a practical understanding of encryption basics, key management and the trade-offs between simplicity and security.

2 Literature Review

Cryptography has a long history, beginning with simple substitution and transposition methods used in ancient times. The Caesar cipher, attributed to Julius Caesar, and the Atbash cipher, originally used with the Hebrew alphabet, are early examples of monoalphabetic substitution. Their main weakness is that each plaintext letter always maps to the same ciphertext letter, so they can be broken by brute force or by frequency analysis, a technique first described by the Arab scholar Al-Kindi. The Vigenère cipher, a polyalphabetic method, was long considered unbreakable until Kasiski and later Friedman showed how the key length and key could be recovered by statistical analysis. Shannon's work on secrecy systems later placed these ideas on a mathematical foundation and showed that a truly random key, as long as the message and used only once (the one-time pad), gives perfect secrecy.

Modern cryptography moved to block and stream ciphers such as DES and AES, and to public-key systems such as RSA and elliptic-curve cryptography, whose security depends on large keys and computationally hard problems rather than on secrecy of the algorithm. XOR-based stream encryption is still central to these designs, and encodings such as Base64 are widely used to carry binary ciphertext through text-based channels such as e-mail and JSON. Educational literature nevertheless continues to teach classical ciphers first, because they let students implement complete encrypt and decrypt cycles with only basic programming constructs, and because their weaknesses clearly show why stronger designs are needed.

Many existing online cipher tools and tutorials implement only a single algorithm, hide the underlying code, or accept only alphabetic input, which limits their learning value. This project addresses these gaps by providing a single Python program that offers five different ciphers through a simple menu, preserves digits, spaces and punctuation, validates keys and modes, and reports errors clearly. Because every cipher has its own encrypt and decrypt function, the code can be read, tested and extended easily. The tool therefore serves as a practical bridge between the theory of classical cryptography and hands-on Python programming for students and beginners.

3 Objectives

- **To design and develop a Python-based text encryption and decryption tool supporting multiple ciphers.**

The primary objective of this project is to build a functional and easy-to-use program in Python that can convert plaintext into ciphertext and recover the original text again. The system will implement five techniques: the Caesar cipher, the Vigenère cipher, a keyed Substitution cipher, the Atbash cipher and an XOR cipher with Base64 encoding. Each technique will be written as a separate, reusable function pair so that the encryption logic and the decryption logic remain independent and easy to test.

- **To implement a menu-driven interface with input validation and error handling.**

A simple console interface is essential to help students and users interact with the tool without difficulty. This objective focuses on creating a menu that lets the user choose the algorithm, select encryption or decryption, and enter the text together with the shift value or key. The program must validate every input, such as non-numeric shifts, empty keys and invalid menu choices, and it must display clear messages instead of crashing. Non-alphabetic characters such as digits, spaces and punctuation are preserved so that messages remain readable after decryption.

- **To verify correctness and compare the strength of each cipher through sample inputs and outputs.**

This objective involves testing every cipher with sample messages and confirming that decrypting the ciphertext with the same key always returns the original text. The outputs of the different algorithms are compared to show how key space, use of a keyword and byte-level operations change the appearance and security of the ciphertext. By observing these results, users can understand the trade-offs between simplicity and security, recognise why classical ciphers are vulnerable to attacks such as brute force and frequency analysis, and appreciate the need for modern encryption standards.

4 Methodology

4.1 Architectural Diagram

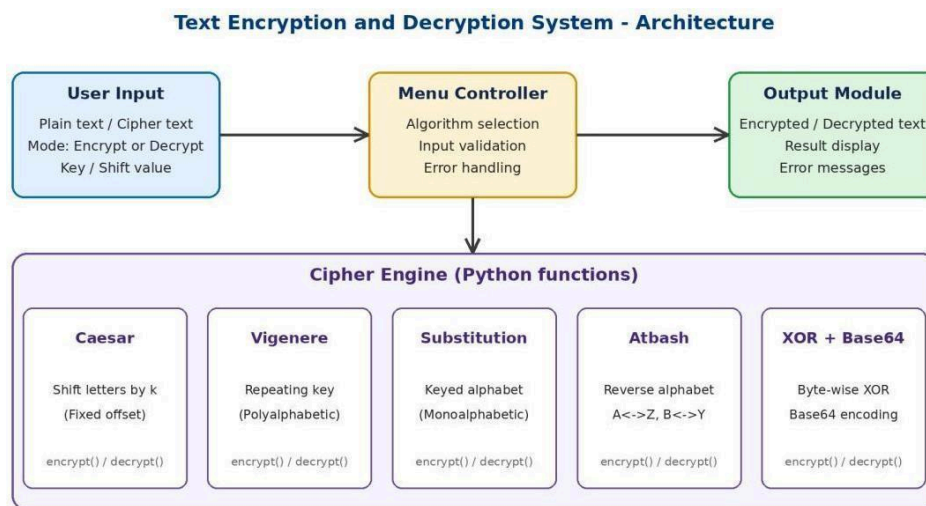


Figure 1: Architectural Diagram

The User Input module collects the message from the user together with the operating mode (encrypt or decrypt) and the shift value or key. The Menu Controller, implemented in the `main()` function, displays the list of algorithms, reads the choice, validates every entry and forwards the request to the correct cipher function. It also handles invalid choices, empty keys and non-numeric shifts using conditions and try-except blocks, so that the program never terminates unexpectedly.

The Cipher Engine forms the core of the system. It contains the Caesar, Vigenère, Substitution, Atbash and XOR + Base64 functions. Each function processes the message character by character or byte by byte using `ord()`, `chr()`, modular arithmetic and the `base64` module, and returns the transformed text. Letters are shifted or replaced while keeping their case, and characters that are not letters are copied without change, which guarantees exact recovery during decryption.

The Output Module receives the result from the cipher engine and displays the encrypted or decrypted text on the console. If an exception is raised, for example when an invalid Base64 string or a wrong key is supplied, the module shows an error message and returns the user to the main menu so that another operation can be attempted.

Together, these components form a modular and extensible framework. New algorithms such as Rail Fence, Playfair or AES-based encryption can be added by writing another function pair and one more menu option, and a graphical or file-based interface can be built later without changing the cipher logic.

4.2 Hardware and Software Requirements

Hardware:

- Intel/AMD Processor
- Minimum 4GB RAM
- Standard Display & Input Devices

Software:

- Python 3.8 or above (Python interpreter)
- Python standard library modules: string and base64
- Any IDE or editor (IDLE, VS Code, PyCharm)

4.3 Working Principle

The working principle of this Text Encryption and Decryption program is based on transforming each character of a message with a mathematical rule controlled by a key. In encryption, the plaintext is converted into ciphertext so that it cannot be understood without the key; in decryption, the inverse rule is applied to recover the original text. For letters, the program converts a character to a number between 0 and 25 using `ord()`, applies the cipher rule with modulo 26 arithmetic, and converts the result back to a letter using `chr()`, keeping the original upper or lower case. Characters that are not letters are copied as they are.

The system operates by first displaying a menu and collecting the algorithm choice, the mode (E for encrypt, D for decrypt), the text and the key. The Caesar cipher uses $E(x) = (x + k) \bmod 26$ and $D(x) = (x - k) \bmod 26$. The Vigenère cipher applies a different shift to each letter based on the repeating keyword. The Substitution cipher builds a keyed alphabet from a key word and maps each letter through it, while the Atbash cipher maps A to Z, B to Y and so on. The XOR cipher converts the text to bytes, XORs every byte with the repeating key, and encodes the result with Base64; decryption reverses these steps. The example below uses the plaintext "Hello World 123".

Example problem:

Algorithm	Key	Plaintext	Ciphertext
Caesar	3	Hello World 123	Khoor Zruog 123
Vigenère	KEY	Hello World 123	Rijvs Uyvjn 123
Substitution	python	Hello World 123	Boffj Vjmfh 123
Atbash	None	Hello World 123	Svool Dliow 123
XOR + Base64	secret	Hello World 123	OwAPHgpUJAoRHgFUQldQ

Caesar Cipher

Rule (shift $k = 3$):

Encryption: $E(x) = (x + 3) \bmod 26$ | Decryption: $D(x) = (x - 3) \bmod 26$

- Example: Hello World 123 $\rightarrow$ Khoor Zruog 123
- Only 25 useful keys, so it can be broken easily by brute force

Vigenère Cipher

Rule (key = KEY):

Encryption: $C = (P + K) \bmod 26$ | Decryption: $P = (C - K) \bmod 26$, key repeated over letters

- Example: Hello World 123 $\rightarrow$ Rijvs Uyvjn 123
- Polyalphabetic, so the same letter can give different ciphertext letters

Substitution Cipher

Rule (key word = python):

Alphabet becomes: pythonabcdefghijklmnopqrstuvwxyz (key word letters first, then remaining letters)

- Example: Hello World 123 $\rightarrow$ Boffj Vjmfh 123
- Huge key space, but still vulnerable to frequency analysis

Atbash Cipher

Rule (no key):

Letter at position x is replaced by the letter at position $25 - x$ ($A \leftrightarrow Z$, $B \leftrightarrow Y$, $C \leftrightarrow X$...)

- Example: Hello World 123 $\rightarrow$ Svool Dliow 123
- Applying the same function twice returns the original text

XOR + Base64 Cipher

Rule (key = secret):

Each byte of the text is XORed with the repeating key bytes; the result is Base64 encoded

- Example: Hello World 123 → OwAPHgpUJAoRHgFUQldQ
- Byte-level cipher; strong only if the key is long, random and used once

4.4 Software Implementation

```
import string

ALPHABET = string.ascii_lowercase

# ----- Caesar Cipher -----
def caesar_encrypt(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

# ----- Vigenere Cipher -----
def vigenere(text, key, decrypt=False):
    result, k = "", 0
    key = [ord(c.lower()) - 97 for c in key if c.isalpha()] if not key:
        raise ValueError("Key must contain at least one letter")
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            s = -key[k % len(key)] if decrypt else key[k % len(key)]
            result += chr((ord(ch) - base + s) % 26 + base)
            k += 1
        else:
            result += ch
    return result

def vigenere_encrypt(text, key):
    return vigenere(text, key, False)

def vigenere_decrypt(text, key):
    return vigenere(text, key, True)

# ----- Substitution Cipher -----
def make_sub_key(seed_word):
    seen = []
    for c in seed_word.lower() + ALPHABET:
        if c.isalpha() and c not in seen:
            seen.append(c)
    return "".join(seen)

def substitution(text, key, decrypt=False):
    src, dst = (key, ALPHABET) if decrypt else (ALPHABET, key)
    table = {}
    for a, b in zip(src, dst):
        table[a] = b
        table[a.upper()] = b.upper()
    return "".join(table.get(ch, ch) for ch in text)

# ----- Atbash -----
```

```

Cipher def atbash(text):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr(base + 25 - (ord(ch) - base))
        else:
            result += ch
    return result

# ----- XOR + Base64 Cipher -----
import base64

def xor_encrypt(text, key):
    data =
    text.encode("utf-8") kb =
    key.encode("utf-8")
    out = bytes(b ^ kb[i % len(kb)] for i, b in enumerate(data))
    return base64.b64encode(out).decode("ascii")

def xor_decrypt(cipher, key):
    data =
    base64.b64decode(cipher) kb =
    key.encode("utf-8")
    out = bytes(b ^ kb[i % len(kb)] for i, b in enumerate(data))
    return out.decode("utf-8")

# ----- Menu Driven Program
-----def main():
    while True:
        print("\n===== TEXT ENCRYPTION & DECRYPTION =====")
        print("1. Caesar Cipher")
        print("2. Vigenere Cipher")
        print("3. Substitution Cipher")
        print("4. Atbash Cipher")
        print("5. XOR + Base64 Cipher")
        print("6. Exit")
        choice = input("Select algorithm: ")
        choice = choice.strip() if choice == "6":
            print("Thank you!")
            break
        if choice not in {"1", "2", "3", "4", "5"}:
            print("Invalid choice. Try again.")
            continue
        mode = input("Encrypt (E) or Decrypt (D)? ")
        mode = mode.strip().upper() if mode not in ("E", "D"):
            print("Invalid mode.")
            continue
        text = input("Enter text: ")
        try:
            if choice == "1":
                s = int(input("Enter shift (integer): "))
                out = caesar_encrypt(text, s) if mode == "E" else caesar_decrypt(text, s)
            elif choice == "2":
                k = input("Enter keyword: ")
                out = vigenere(text, k, mode == "D")
            elif choice == "3":
                k = make_sub_key(input("Enter key word: "))
                out = substitution(text, k, mode == "D")
            elif choice == "4":
                out = atbash(text)
            else:
                k = input("Enter secret key: ")
                if not k:
                    raise ValueError("Key cannot be empty")
                out = xor_encrypt(text, k) if mode == "E" else xor_decrypt(text, k)
            print("Result :", out)
        except Exception as ex:
            print("Error:", ex)

if __name__ == "__main__":
    main()

```

5 Results and Discussion

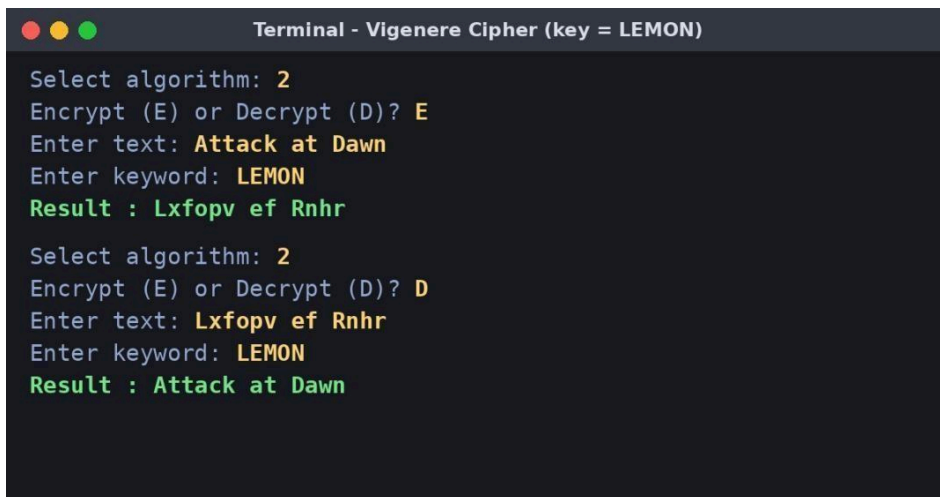


```
Terminal - Caesar Cipher (shift = 3)

Select algorithm: 1
Encrypt (E) or Decrypt (D)? E
Enter text: Hello World
Enter shift (integer): 3
Result : KhooR Zruog

Select algorithm: 1
Encrypt (E) or Decrypt (D)? D
Enter text: KhooR Zruog
Enter shift (integer): 3
Result : Hello World
```

Figure 2: Output (Caesar Cipher)



```
Terminal - Vigenere Cipher (key = LEMON)

Select algorithm: 2
Encrypt (E) or Decrypt (D)? E
Enter text: Attack at Dawn
Enter keyword: LEMON
Result : Lxfopv ef Rnhr

Select algorithm: 2
Encrypt (E) or Decrypt (D)? D
Enter text: Lxfopv ef Rnhr
Enter keyword: LEMON
Result : Attack at Dawn
```

Figure 3: Output (Vigenère Cipher)



```
Terminal - Substitution and Atbash Ciphers

Select algorithm: 3
Encrypt (E) or Decrypt (D)? E
Enter text: Hello World
Enter key word: python
Result : Boffj Vjmfh

Select algorithm: 4
Encrypt (E) or Decrypt (D)? E
Enter text: Hello World
Result : Svool Dliow
```

Figure 4: Output (Substitution and Atbash Ciphers)

A terminal window titled "Terminal - XOR + Base64 Cipher (key = secret)" with a dark background. It shows two interactions. In the first, the user selects algorithm 5, chooses to encrypt (E), enters the text "Meet at 5 PM" and the key "secret", and the result is "PgAGBkUVB0VWUjU5". In the second, the user selects algorithm 5, chooses to decrypt (D), enters the text "PgAGBkUVB0VWUjU5" and the key "secret", and the result is "Meet at 5 PM".

```
Terminal - XOR + Base64 Cipher (key = secret)

Select algorithm: 5
Encrypt (E) or Decrypt (D)? E
Enter text: Meet at 5 PM
Enter secret key: secret
Result : PgAGBkUVB0VWUjU5

Select algorithm: 5
Encrypt (E) or Decrypt (D)? D
Enter text: PgAGBkUVB0VWUjU5
Enter secret key: secret
Result : Meet at 5 PM
```

Figure 5: Output (XOR + Base64 Cipher)

Summary of Results

All five ciphers encrypt and decrypt correctly. Caesar turned "Hello World" into "Khoor Zruog", Vigenère with key LEMON turned "Attack at Dawn" into "Lxfopv ef Rnhr", and the XOR cipher produced a Base64 string that decrypted back to "Meet at 5 PM". In every test the original text was recovered exactly, with digits, spaces and punctuation preserved.

Interpretation

Caesar and Atbash are the weakest, since Caesar has only 25 keys and Atbash has none. Substitution has a large key space but keeps letter frequencies, so frequency analysis breaks it. Vigenère hides frequencies better but is weak with a short key. XOR works on bytes and is secure only with a long, random, single-use key.

Achievement of Objectives

The project met its goals by implementing five ciphers in Python, a menu-driven interface with input validation, and correct recovery of the original message during decryption.

Implications

The program can be used to teach cryptography and Python basics, and as a base for adding stronger algorithms such as AES or RSA.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied Python programming and cryptography fundamentals to build a working encryption and decryption tool.
PO2	Problem Analysis	Analysed how classical ciphers work and fail (brute force, frequency analysis) to select suitable algorithms.
PO3	Design/Development of Solutions	Designed a menu-driven program with separate encrypt and decrypt functions, validation and error handling.
PO4	Conduct Investigations of Complex Problems	Tested each cipher with varied inputs and keys to confirm that decryption recovers the original text.
PO5	Engineering Tool Usage	Used Python, its standard library (string, base64) and IDEs such as IDLE or VS Code to build and debug the tool.
PO8	Individual and Collaborative Team work	Divided algorithms, coding, testing and documentation among team members with regular reviews.
PO9	Communication	Presented the design, code and outputs clearly through this report, diagram and screenshots.
PO11	Life-Long Learning	Explored cryptography beyond the syllabus, including Vigenère, XOR and Base64, and ways to extend the tool.

6.2 Program Specific Outcomes (PSOs) Mapping

Table 2: Mapping of Program Specific Outcomes (PSOs) to Project Relevance

PSO No.	Program Specific Outcome	Relevance
PSO1	Software Development & Fundamentals	Applied core Python concepts, data structures, and modular function design to implement multiple encryption algorithms.
PSO2	Information Security & Systems	Demonstrated fundamental cryptographic principles and byte-level operations to ensure data confidentiality and integrity.

6.3 Sustainable Development Goals (SDGs) Mapping

Table 3: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Helps students learn cryptography and Python through hands-on, visible examples.
SDG 9	Industry, Innovation and Infrastructure	Promotes secure software design and innovation for trustworthy digital infrastructure.
SDG 8	Decent Work and Economic Growth	Builds programming and security skills that improve employability in IT careers.

7 Conclusion

This project successfully implemented a Python-based Text Encryption and Decryption tool that supports the Caesar, Vigenère, Substitution, Atbash and XOR + Base64 ciphers. Through a simple menu-driven interface, the system lets users enter text, choose an algorithm, provide a key and obtain the encrypted or decrypted output instantly, while validating inputs and handling errors gracefully. The results show that every cipher recovers the original message exactly, and the comparison of the algorithms highlights how key size, use of a keyword and byte-level operations affect the strength of encryption. The project effectively bridges the theory of classical cryptography with practical Python programming.

For future enhancements, the tool can be extended with more algorithms such as Rail Fence, Playfair, Hill and one-time-pad ciphers, and with modern standards such as AES and RSA using libraries like cryptography or PyCryptodome. Features such as file encryption, password-based key generation, a graphical user interface using Tkinter, a web or mobile version, and automatic cipher-breaking demonstrations would further increase its educational value. Continuing development in these areas will strengthen the tool and expand its usefulness in academic and practical security applications.

Bibliography

Online Resources

- [1] Python Software Foundation – Python 3 Documentation: base64 module <https://docs.python.org/3/library/base64.html>
- [2] GeeksforGeeks – Caesar Cipher in Cryptography <https://www.geeksforgeeks.org/caesar-cipher-in-cryptography/>
- [3] Wikipedia – Vigenère Cipher https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher
- [4] Python Software Foundation – The Python Tutorial <https://docs.python.org/3/tutorial/>
- [5] Wikipedia – Atbash Cipher <https://en.wikipedia.org/wiki/Atbash>

Research article references

- [1] Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson Education.
- [2] Forouzan, B. A. (2015). Cryptography and Network Security (2nd ed.). McGraw-Hill Education.
- [3] Shannon, C. E. (1949). Communication Theory of Secrecy Systems. Bell System Technical Journal, 28(4), 656–715.
- [4] Sweigart, A. (2013). Hacking Secret Ciphers with Python. CreateSpace Independent Publishing Platform.
(Practical Python implementations of classical ciphers, relevant to the project methodology)